

GUIA PRÁTICO

PREVISÃO DE FALHAS NO PROJETO FINAL

Do problema preditivo à implementação, avaliação e uso da IA generativa

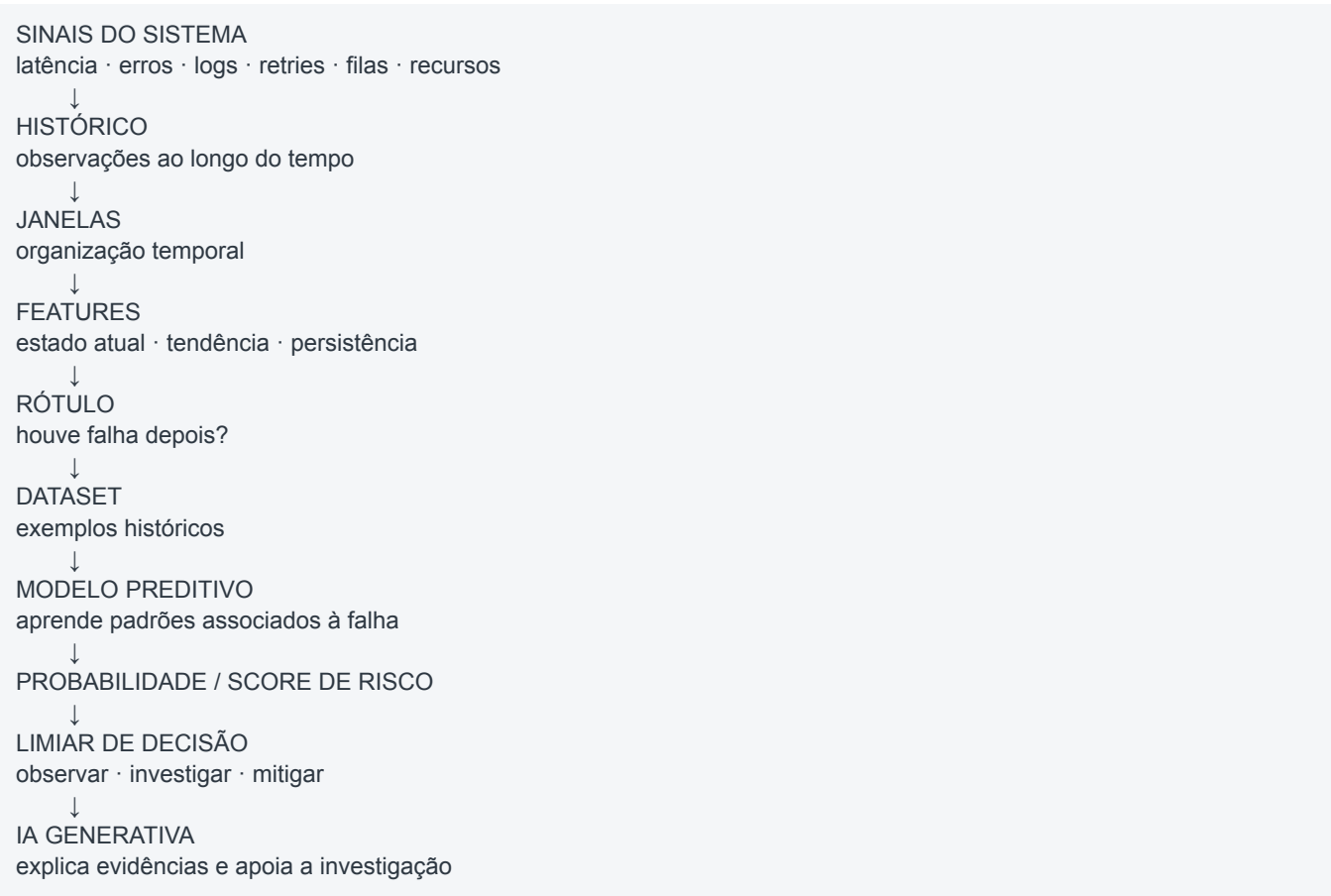
Objetivo do material

Ajudar você a adaptar o método do laboratório do Dia 3 ao seu próprio projeto. O foco não é copiar as métricas do professor, mas entender como definir a falha, organizar sinais em janelas, criar features e rótulos, treinar um modelo, avaliar o resultado e usar IA generativa apenas para interpretar a previsão.

MÓDULO 2 • SEMANA 10

1. O que você vai construir

A previsão de falhas é um pipeline de engenharia. O modelo é apenas uma das etapas. O trabalho começa definindo claramente o que deve ser antecipado e quais sinais do sistema podem carregar evidência dessa falha.



Princípio fundamental

O modelo preditivo calcula o risco. A IA generativa interpreta o resultado. A IA não deve recalcular probabilidade, redefinir limiar, alterar features ou substituir a decisão técnica.

2. Antes de usar o prompt: defina o problema

Antes de abrir o Claude Code ou outra ferramenta de implementação, preencha as quatro decisões abaixo. Elas orientam todo o restante do pipeline.

Pergunta	O que você deve definir	Exemplo
Qual operação quero acompanhar?	Uma operação importante do seu sistema.	checkout, upload, geração de relatório, execução de agente
Qual falha quero antecipar?	Um evento objetivo que represente falha.	timeout, HTTP 500, fila saturada, execução incompleta
Qual horizonte faz sentido?	Quanto tempo ou quantas execuções à frente você deseja prever.	próximas 5 execuções, próximos 10 minutos

Pergunta	O que você deve definir	Exemplo
Quais sinais tenho disponíveis?	Métricas ou eventos que possam anteceder a falha.	latência, erros, retries, logs, CPU, filas

Contexto do meu projeto

Operação monitorada:

Falha que quero prever:

Horizonte desejado:

Sinais disponíveis:

3. Preparar o repositório

Crie uma branch específica antes de iniciar a implementação. Isso mantém o experimento isolado e facilita revisar o que foi alterado.

```
git switch main
git pull
git switch -c feat/predictive-failure

git status
git branch --show-current
```

Depois, abra o assistente de implementação no diretório do projeto:

```
claude
```

Importante

Cole o prompt e aguarde o plano. Não aprove uma implementação que você ainda não compreendeu. O plano deve explicar o que será previsto, de onde virão os dados, quais features serão calculadas, como os rótulos serão criados e como o modelo será avaliado.

4. Prompt-base para o projeto final

O prompt abaixo é propositalmente genérico. Ele deve funcionar como base em projetos diferentes. Substitua os campos entre colchetes pelo contexto do seu sistema.

Você é um engenheiro de software com experiência em observabilidade, Machine Learning e previsão de falhas.

Analise o repositório atual antes de realizar qualquer alteração.

CONTEXTO DO MEU PROJETO

Operação monitorada:
[DESCREVA]

Falha que quero antecipar:
[DESCREVA]

Horizonte desejado:
[DESCREVA]

Sinais disponíveis:
[DESCREVA]

OBJETIVO

Implemente uma primeira versão de um pipeline de previsão de falhas adaptado a este projeto.

Antes de implementar:

1. identifique a stack e a arquitetura atual;
2. identifique onde os sinais informados podem ser coletados;
3. avalie se existem dados históricos suficientes;
4. proponha:
 - definição objetiva da falha;
 - horizonte de previsão;
 - organização das observações em janelas;
 - features de estado atual, tendência e persistência;
 - estratégia de criação dos rótulos;
 - modelo preditivo simples e interpretável;
 - estratégia de divisão entre treino e teste;
 - métricas de avaliação;
 - limiar inicial de decisão.

Apresente esse plano antes de alterar o código.
Não implemente nada até apresentar o plano.

IMPLEMENTAÇÃO

Depois da aprovação do plano, implemente:

dados históricos
→ janelas
→ extração de features
→ rótulos
→ dataset
→ divisão treino/teste
→ treinamento
→ avaliação
→ previsão
→ probabilidade de falha
→ classificação por limiar.

Evite qualquer vazamento de informação futura.
Comece com um modelo simples e interpretável.
Não use modelos complexos sem necessidade justificada.

Se não houver histórico suficiente, crie dados sintéticos somente para demonstrar o pipeline e deixe claramente identificado que são simulados.

A previsão deve apresentar pelo menos:
- probabilidade ou score de risco;

- classificação baseada em um limiar;
- principais fatores relacionados à previsão;
- janela ou dados utilizados na inferência.

Se houver IA generativa no projeto, ela pode produzir:

- evidências;
- interpretação;
- hipóteses;
- recomendações;
- limitações.

A IA generativa não pode calcular nem alterar:

- probabilidade;
- score de risco;
- classificação;
- limiar;
- features calculadas pelo pipeline.

INTERFACE DE EXECUÇÃO

Se o projeto utilizar Node.js e package.json, crie, quando fizer sentido, os seguintes scripts:

predictive:prepare

- prepara os dados, gera features, cria os rótulos e treina o modelo.

predictive:evaluate

- avalia o modelo usando somente dados não utilizados no treinamento.

predictive:predict

- executa uma previsão utilizando uma nova sequência ou conjunto de sinais.

predictive:scenario

- executa um cenário controlado para demonstrar degradação ou recuperação.

test:predictive

- executa os testes específicos do pipeline preditivo.

Se a stack não utilizar npm, crie comandos equivalentes seguindo os padrões existentes no projeto.

TESTES

Garanta principalmente que:

- nenhuma feature utilize informações futuras;
- treino e teste estejam separados corretamente;
- o treinamento seja reproduzível;
- a inferência seja reproduzível;
- cenários progressivos aumentem o risco de forma coerente;
- recuperação possa reduzir o risco;
- a IA generativa não altere a previsão;
- nenhuma credencial apareça nos relatórios.

DOCUMENTAÇÃO

Ao finalizar, explique:

1. arquitetura criada;
2. arquivos criados e alterados;
3. responsabilidade de cada arquivo;
4. sinais utilizados;
5. features utilizadas;
6. definição da falha;
7. horizonte;
8. estratégia de rotulagem;
9. modelo utilizado;

10. limiar escolhido;
11. como executar treinamento;
12. como executar avaliação;
13. como realizar uma nova previsão;
14. como alterar sinais, features, horizonte e limiar.

Execute os testes e apresente os resultados.

Não faça commit, push ou Pull Request.
 Não adicione infraestrutura ou dependências complexas sem necessidade.
 Reutilize a stack e os padrões já existentes no projeto.

5. O que o prompt faz

O prompt não pede apenas “crie uma IA para prever falhas”. Ele força a implementação a passar pelas decisões que formam um problema preditivo válido.

Etapa	Pergunta respondida
Inspeção	Como o projeto está organizado e onde a solução deve se integrar?
Falha	Qual evento futuro o modelo tentará antecipar?
Horizonte	Quanto tempo ou quantas execuções à frente serão consideradas?
Sinais	Quais observações podem carregar evidência preditiva?
Janelas	Como as observações serão organizadas temporalmente?
Features	Como o comportamento será convertido em números?
Rótulos	Como saberemos, nos exemplos históricos, se houve falha depois?
Dataset	Como serão organizados os exemplos de treino e avaliação?
Modelo	Como as features serão convertidas em probabilidade ou score?
Avaliação	Como verificar se o modelo generaliza e quais erros comete?
Limiar	Quando o risco passa de observação para investigação ou mitigação?
IA generativa	Como explicar a previsão sem alterar o resultado numérico?

6. Validar o plano antes de implementar

Quando a ferramenta apresentar o plano, confirme se ele responde às perguntas abaixo. Se alguma resposta estiver vaga, peça ajuste antes de permitir mudanças no repositório.

- Qual falha será prevista?
- Qual é o horizonte de previsão?
- Quais sinais serão usados?
- Como será formada uma janela?

- Quais features serão calculadas?
- Como serão criados os rótulos?
- De onde virá o histórico?
- Como treino e teste serão separados?
- Qual modelo simples será utilizado e por quê?
- Quais métricas serão usadas na avaliação?
- Qual limiar inicial será adotado?
- Como a IA generativa ficará separada do cálculo preditivo?

Se não houver histórico real

Dados sintéticos podem ser utilizados para demonstrar o ciclo técnico, mas isso deve aparecer explicitamente como limitação. Um modelo treinado apenas com dados simulados não comprova capacidade preditiva em produção.

7. Depois da implementação: descubra o que foi criado

Antes de executar qualquer treinamento, revise as alterações e identifique os comandos realmente disponíveis.

```
git status
git diff --stat
```

```
# Em projetos Node.js:
npm run
```

Se o prompt tiver sido seguido em um projeto Node.js, os scripts podem ficar próximos desta convenção:

Comando sugerido	Responsabilidade
npm run predictive:prepare	preparar histórico/dataset, extrair features, criar rótulos e treinar o modelo
npm run predictive:evaluate	avaliar o modelo em dados não usados no treinamento
npm run predictive:predict	realizar uma nova previsão
npm run predictive:scenario -- <nome>	executar um cenário controlado de degradação ou recuperação
npm run test:predictive	executar os testes específicos do pipeline preditivo

Atenção

Esses nomes são uma convenção sugerida, não uma garantia. Use `npm run` para confirmar os scripts criados no seu projeto. Em outra stack, use os comandos equivalentes definidos pela implementação.

8. Preparar dados e treinar o modelo

```
npm run predictive:prepare
```

Esse comando deve executar, conceitualmente, o seguinte percurso:

```
histórico
↓
janelas
↓
features
↓
rótulos
↓
dataset
↓
treinamento
↓
modelo persistido
```

Ao final, procure informações como quantidade de exemplos, features utilizadas, proporção treino/teste, distribuição de positivos e negativos e caminho do modelo salvo.

9. Avaliar o modelo

```
npm run predictive:evaluate
```

A avaliação deve acontecer em dados que não participaram do treinamento. Verifique, quando aplicável:

- **Accuracy:** visão geral de acertos, mas não deve ser usada isoladamente.
- **Precision:** das previsões positivas, quantas realmente antecederam falha?
- **Recall:** das falhas futuras, quantas foram antecipadas?
- **F1:** equilíbrio entre precision e recall.
- **Falso positivo:** o modelo alertou, mas a falha não ocorreu no horizonte.
- **Falso negativo:** a falha ocorreu, mas o modelo não alertou.

Pergunta de projeto

Qual erro custa mais no seu contexto: investigar sem necessidade ou deixar uma falha importante passar sem alerta? Essa resposta ajuda a justificar o limiar e as métricas priorizadas.

10. Executar uma nova previsão

```
npm run predictive:predict
```

A saída deve deixar separados o resultado do modelo e a interpretação. Um exemplo possível:

```
Probabilidade de falha: 0.78
Limiar: 0.70
Classificação: RISCO ALTO
```

```
Principais fatores:
- latencySlope
- errorRate
- consecutiveDegradedWindows
```

O raciocínio é:


```

features
  ↓
modelo
  ↓
0.78

0.78 >= 0.70
  ↓
MITIGAR

```

11. Executar cenários controlados

Cenários controlados ajudam a verificar se a trajetória do risco faz sentido. Use nomes e dados coerentes com o seu domínio.

```

npm run predictive:scenario -- degradation
npm run predictive:scenario -- recovery

```

Cenário	Leitura esperada
Degradação progressiva	o risco tende a crescer à medida que tendência e persistência se fortalecem
Recuperação	o risco deve recuar quando as janelas seguintes mostram melhora consistente
Pico isolado	pode gerar atenção temporária, mas não deve necessariamente permanecer em risco alto
Estabilidade	o risco deve permanecer baixo ou coerente com a referência do projeto

Os números não precisam ser iguais aos do laboratório do professor. O que deve ser coerente é a relação entre histórico, features, probabilidade e resultado.

12. Executar os testes específicos

```
npm run test:predictive
```

Os testes devem proteger principalmente contra erros de metodologia e integração:

- nenhuma feature utiliza informação futura;
- treino e teste estão separados corretamente;
- treinamento e inferência são reproduzíveis;
- cenários de degradação aumentam o risco de forma coerente;
- cenários de recuperação reduzem o risco quando apropriado;
- a IA generativa não altera probabilidade, classificação ou limiar;
- credenciais e segredos não aparecem nos relatórios.

13. Validar o projeto inteiro

Depois de validar o módulo preditivo, execute também as verificações já existentes no seu projeto.

```
npm run lint
npm test
npm run build
```

```
# Se existir um comando agregado:
npm run ci
```

Critério

A nova funcionalidade não pode quebrar o projeto existente. O pipeline preditivo deve ser uma extensão integrada ao sistema, não uma arquitetura paralela que ignora os padrões já utilizados.

14. Inspecionar e explicar o código

Na apresentação final, não basta dizer que a ferramenta implementou. Você deve localizar no código onde cada responsabilidade está implementada.

O que localizar	Pergunta que você deve responder
Histórico/coleta	De onde vêm os dados utilizados?
Janelas	Como o tempo ou as execuções são agrupados?
Features	Como o comportamento vira números?
Rótulo/target	Como o sistema determina se houve falha futura?
Treinamento	Onde o modelo aprende?
Avaliação	Onde são calculadas as métricas?
Predição	Onde a probabilidade ou score é calculado?
Limiar	Onde o risco se transforma em uma ação?
Explicação	Onde a IA generativa recebe e interpreta o resultado?

15. Faça pelo menos uma alteração manual

Depois da primeira versão, altere conscientemente uma decisão do pipeline e compare o efeito. Essa etapa demonstra domínio do mecanismo e não apenas execução de código gerado.

- alterar o horizonte;
- alterar o limiar;
- adicionar ou remover uma feature;
- alterar a definição de falha;
- alterar o tamanho das janelas;
- alterar um cenário de degradação ou recuperação.

Depois da mudança, execute novamente as etapas afetadas. Exemplo:

```
npm run predictive:prepare
npm run predictive:evaluate
npm run predictive:scenario -- degradation
```

Registre o que mudou nas probabilidades, métricas ou classificações e explique o motivo.

16. Onde a IA generativa entra

MODELO PREDITIVO



probabilidade = 0.82

limiar = 0.70



RISCO ALTO



IA GENERATIVA

explica evidências, hipóteses,
recomendações e limitações



HUMANO

investiga e decide

Evite a arquitetura abaixo:

dados



LLM



"82% de chance de falha"

Regra de confiabilidade

A LLM não deve inventar o score. O cálculo deve vir do modelo preditivo ou de uma regra quantitativa explícita. A IA entra depois, como camada de interpretação.

17. Laboratório do professor: comandos de referência

No laboratório da Semana 10, Dia 3, os scripts de referência são estes:

```
npm run test:day3
npm run day3:prepare
```

```
npm run day3:scenario -- latency-growth
npm run day3:scenario -- error-growth
npm run day3:scenario -- log-growth
npm run day3:scenario -- transient-spike
npm run day3:scenario -- all
```

```
npm run day3:challenge -- case-a
npm run day3:challenge:reveal -- case-a
npm run day3:challenge -- case-a --mitigated
```

Use esses comandos como referência de organização, não como nomes obrigatórios para o seu projeto.

Laboratório do professor	No seu projeto
day3:prepare	predictive:prepare ou equivalente
day3:scenario	predictive:scenario ou equivalente

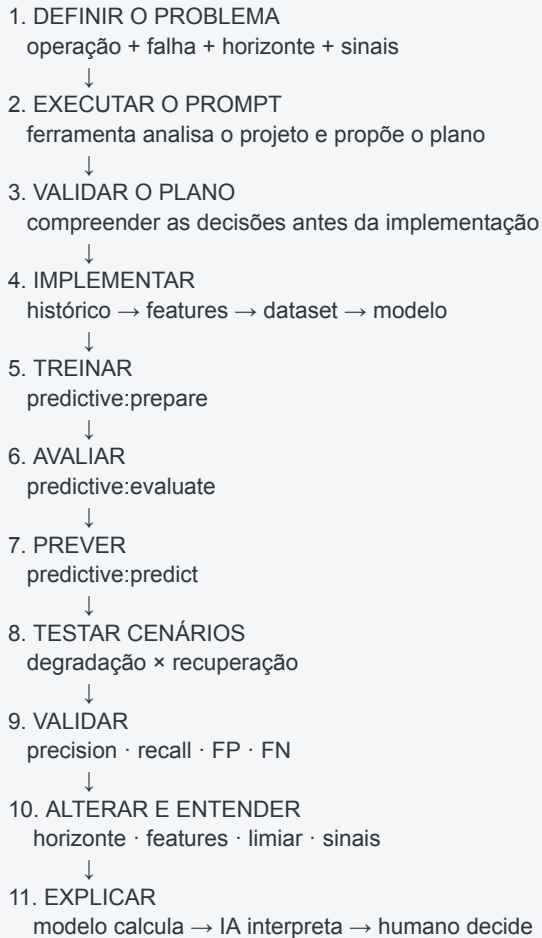
Laboratório do professor	No seu projeto
latency/error/log-growth	cenários coerentes com o seu domínio
métricas do laboratório	sinais reais do seu projeto
falha definida pelo professor	falha objetiva definida por você
horizonte de duas janelas	horizonte justificado para o seu sistema

18. O que apresentar no projeto final

A apresentação deve demonstrar raciocínio técnico. Prepare-se para responder, mostrando código e execução:

1. Qual falha você tentou antecipar?
2. Por que essa falha é relevante?
3. Qual horizonte foi escolhido?
4. Quais sinais foram utilizados?
5. Como esses sinais foram organizados em janelas?
6. Quais features foram criadas?
7. Como os rótulos foram produzidos?
8. De onde veio o histórico?
9. Como treino e teste foram separados?
10. Qual modelo foi utilizado e por quê?
11. Como o modelo foi avaliado?
12. Qual limiar foi adotado?
13. Mostre um caso em que o risco aumentou.
14. Mostre um caso de recuperação ou não falha.
15. Onde a IA generativa entra?
16. Qual é a principal limitação da solução?

19. Fluxo final recomendado



20. Checklist de conclusão

- Consigo definir claramente a falha que desejo antecipar.
- Consigo justificar o horizonte de previsão.
- Consigo explicar quais sinais alimentam o pipeline.
- Consigo localizar e explicar as features.
- Consigo explicar como os rótulos são criados.
- Consigo explicar a origem do histórico e suas limitações.
- Consigo mostrar como treino e teste foram separados.
- Consigo interpretar precision, recall, falso positivo e falso negativo.
- Consigo executar uma nova previsão e explicar o limiar.
- Consigo mostrar um cenário de degradação e um de recuperação.
- Consigo provar que a IA generativa não altera o resultado preditivo.
- Fiz pelo menos uma alteração manual e analisei seu efeito.
- Consigo explicar uma limitação relevante da solução.

Condição de conclusão

Você terminou quando consegue mostrar, no próprio projeto: qual falha quer antecipar, quais sinais observa, como transforma o histórico em features e rótulos, onde o modelo é treinado, como calcula o risco, como o limiar orienta a ação e como a IA interpreta o resultado sem alterá-lo.

Observações finais

Os exemplos de comandos deste guia assumem um projeto Node.js com npm. Em projetos Python, Java, .NET ou outras stacks, crie comandos equivalentes que respeitem a arquitetura existente. O método permanece o mesmo: definir problema, organizar dados temporais, construir features e rótulos, treinar, avaliar, prever, interpretar e validar.

Dados sintéticos são adequados para demonstrar o ciclo técnico, mas não substituem histórico real de telemetria e incidentes quando a intenção é avaliar uma solução de produção.